

Experiment 6 — Automatic packed arrays

Mathilda — execution-speed experiments

Contents

Experiment 6 — Automatic packed arrays	1
Hypothesis	1
The contract	1
What was built	2
Results	2
What switching it on exposed	3
The finding that outlived the experiment	3
Verification	3
Still open	3
Why Mathilda is not the fastest here, and what it would take	4

Experiment 6 — Automatic packed arrays

Date: 2026-07-30 · Commit: 1c8562d · Code: `src/pack.{c,h}`, `src/ndarray.c`, `src/ndstruct.c`, `src/ndreduce.c` · Design: [docs/design/packed_arrays.md](https://mathilda.github.io/docs/design/packed_arrays.md) · Result: 5.9× to 56000×, invisibly

Common method in [README.md](https://mathilda.github.io/README.md).

Hypothesis

`{1., 2., 3.}` in Mathilda was a `List` expression holding three heap-allocated `Expr` nodes. At 10^6 elements that is 10^6 allocations, 10^6 pointer chases per pass, and no chance of the compiler vectorising anything.

Mathematica solved this in version 4 with packed arrays: a dense rectangular list of uniformly-typed machine numbers is stored as a flat buffer, and the change is invisible — the only observable difference is `DeveloperPackedArrayQ`.

Mathilda already had the buffer type (`EXPR_NDARRAY`, built for `Compile[]` and `NDSolve`). The hypothesis was that it could be made automatic without changing a single answer.

The contract

This is the part that mattered more than any speed number.

Packing may never change an answer. Not a value, not an element's head, not a result's structure.

That rules out the obvious cheap version. A buffer is uniformly typed, so packing a list of mixed exact and inexact numbers would have to coerce one of them — and `{1, 2.}` becoming `{1., 2.}` is a wrong answer, not a representation choice. The packer therefore declines on: mixed Integer/Real, Rational, `BigInt`, `MPFR`, Complex, symbolic, ragged, and empty.

The threshold is 250 elements — chosen for blast radius, not for break-even. Break-even is far lower; 250 keeps the change away from the small lists that appear in symbolic code, where the risk of an observable difference is highest and the benefit is nil.

What was built

Nine producers hand back a packed list, by one of two mechanisms:

- Direct construction — the producer writes machine values into the buffer and never builds the nodes at all: `Range` (both branches), `ConstantArray`, `RandomReal`, and `Table` when its body compiles. This is where the large factors come from: packing `Range[1., 106]` after building 10⁶ nodes costs 340 ms + 52 ms; writing 10⁶ doubles costs under 1 ms.
- Offer after building — `pack_offer` on an already-built list, for producers that cannot know the element type in advance: `Table`'s other branches, `Array`, `RandomInteger`, `Sort`, `Select`, and the iterate history of `NestList`/`FoldList`/`NestWhileList`/`FixedPointList`. Declining is $O(1)$ on anything symbolic — the probe allocates nothing and stops at the first element that rules packing out.

The transparency gate in `evaluate_step` is what makes it safe. Roughly 7100 sites in the tree read `data.function.args` directly, and none of them know about buffers. Before any head not on an explicit aware list, the gate materialises every packed argument back into a `List`. Correct by construction; slow by construction; and the aware list is then the entire optimisation surface.

Results

10⁶-element list, packing on against `MATHILDA_NO_PACK=1` — same binary:

expression	packed	plain	
<code>Do[Length[x], {20}]</code>	0.01 ms	538 ms	54000×
<code>Do[x[[7]], {20}]</code>	0.01 ms	558 ms	56000×
<code>x . x (integer)</code>	0.95 ms	448 ms	471×
<code>2 x, x + 1 (integer)</code>	1.8 ms	330 ms	~185×
<code>Range[10⁶]</code>	0.84 ms	113 ms	135×
<code>Total[x] (integer)</code>	0.82 ms	93 ms	114×
<code>Accumulate[x]</code>	5.2 ms	377 ms	73×
<code>Total[x] (real)</code>	1.4 ms	92 ms	67×
<code>Sin[x]</code>	27.6 ms	334 ms	12×
<code>Sort[RandomReal[1, 10⁶]]</code>	158 ms	932 ms	5.9×

The two 5×10⁴ rows are the honest shape of the win: `Length` and `Part` are $O(1)$ on a buffer and $O(1)$ on a list too — what they were paying was the materialisation, over and over, because the head was not aware. The speedup is not "buffers are fast"; it is "not converting is fast".

Against Mathematica and NumPy

Both CAS pack automatically by default, so this is like against like.

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Total (reduction)	2.95 ms	2.29 ms	4.05 ms	1/1.29x	1.37x
Accumulate (prefix scan)	19.42 ms	15.13 ms	28.73 ms	1/1.28x	1.48x
Sort	318.55 ms	714.17 ms	155.76 ms	2.24x	1/2.05x
Sin (elementwise)	13.53 ms	11.71 ms	52.68 ms	1/1.16x	3.89x
Exp (elementwise)	13.37 ms	11.50 ms	54.86 ms	1/1.16x	4.10x
STREAM triad, $a = b + 3c$	33.30 ms	31.82 ms	29.62 ms	1/1.05x	1/1.12x
Dot (inner product)	5.81 ms	5.22 ms	6.90 ms	1/1.11x	1.19x

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Differences	16.28 ms	14.33 ms	15.60 ms	1/1.14x	1/1.04x
Reverse	33.05 ms	11.29 ms	15.76 ms	1/2.93x	1/2.10x
RotateLeft	38.69 ms	10.09 ms	16.54 ms	1/3.83x	1/2.34x

Sin and Exp are 3.9× and 4.1× faster than NumPy because Mathilda threads the elementwise kernel and NumPy's ufuncs are serial; Sort is 2.2× faster than Mathematica (an 8-pass LSD radix sort on the double bit patterns against a comparison sort) and 2.1× behind NumPy's. Reverse and RotateLeft are the remaining serial-buffer band.

What switching it on exposed

Turning packing on did not cause bugs so much as reveal paths that had never run on a buffer. The largest class:

A fast path not on the aware list never runs. Fourier had a complete NDAarray implementation and the gate materialised its argument before the builtin could see it: `Fourier[packedList]` measured 986 ms against `Fourier[NDAarray[...]]` 62.8 ms on identical data. The same happened to Nest, and later to Outer. `make-check-packed-aware` now catches it: every head with a registered ND fast path must opt in or be exempt with a stated reason.

Exactness at the boundary. `Range[10]/2` is a list of Rationals; Total on an int64 buffer accumulating through double is wrong past 2^{53} and returns the wrong head. Before the exact int64 arithmetic of [MACHINE_INTEGERS.md](#), the gate had to materialise every integer buffer — and `Total[Range[106]]` measured 1.55× slower packed than plain.

A DownValue that binds opaquely. `f[x_] := body` binds the whole value and substitutes it, so it reads no element and is safe on a buffer. Without that exemption an integer grid materialised at every helper call, which is the entire vectorised Game of Life benchmark: 65.7 s → 260 ms (253×).

Four leaks, found by a differential valgrind sweep rather than by a test.

The finding that outlived the experiment

Two later sweeps found the same defect eleven times between them:

The packing decision is made about one value in isolation; the cost is paid in proportion to another.

`PACK_MIN_ELEMENTS` judges each value alone — correctly. But a binary operation's cost is set by its largest operand, so a 32-element vector, correctly left unpacked, dragged a 6.4-million-element matrix onto the symbolic path with it. See [HPC_SWEEP_2_APPLICATIONS.md](#) and [HPC_SWEEP_3_NUMPY_GAP.md](#).

Verification

- `tests/test_packed_list.c` — the invisibility invariant, differentially: for a large corpus of expressions, the packed form and the plain form must produce byte-identical printed output, heads included.
- Every case the packer must decline is tested as explicitly as the cases it takes.
- `$AutoArrayPacking` and `MATHILDA_NO_PACK` exist so any doubt can be settled by running both.
- `tests/bench_pack` guards against an aware head silently becoming unaware.

Still open

- Complex does not pack (it would need a presentation-aware zero-imaginary fold to round-trip).
- Packing a large plain list is now itself a measurable cost in the paths that lift a small operand — see HPC plan Phase 8.
- `ArrayReshape` is not implemented at all, so reshaping a buffer — a metadata change — is unavailable.

Why Mathilda is not the fastest here, and what it would take

Packing turned ten operations from $5.9\times$ to $56000\times$ faster within Mathilda. Against the other two systems the picture afterwards is:

row	Mathilda	best other	gap	cause
Sort, 10^7	319 ms	156 ms (NumPy)	2.05×	a comparison sort
Reverse, 10^7	33.1 ms	11.3 ms (WL)	2.93×	serial memcpy per
RotateLeft, 10^7	38.7 ms	10.1 ms (WL)	3.83×	serial, and two pa
Total, Accumulate, Dot, Sin, Exp, triad, Differences	—	—	1.04–1.29×	at or near the mer

The last group is within 30% of both systems and most of it is bandwidth; it is not where the work is. Three rows are.

The road to fastest

1. Thread **Reverse** and **RotateLeft**. Both are pure block moves — `nd_parallel_for` already exists and every elementwise kernel uses it, and neither has any ordering subtlety to worry about. Expected: 33–39 ms $\rightarrow$ ~12 ms, level with Mathematica.
2. Rotate in one pass. `nd_rotate_axes` copies through a scratch buffer, so a rank-1 rotate reads and writes 80 MB twice. For axis 0 it is two memcpys into the destination directly, with no scratch at all.
3. A better sort. 319 ms for 10^7 doubles is ~32 ns/element; NumPy's introsort does it in 15.6. A radix sort on the IEEE bit pattern is the standard answer for float64 keys and is typically 2–3× a comparison sort, which would put this row ahead of both. The int64 arm can share it.
4. A vector math library for **Sin/Exp** — see experiment 2's roadmap; worth ~1.9× on those two rows and applies system-wide.

Items 1–3 are all self-contained kernel work on paths that already exist.